

Ques. What is Generator Functions?

- Generator Function is a function which returns values one by one and saves memory.
- Normal function uses **return**, while generator function uses **yield**.
- The **difference** between **yield** and **return** is that **yield returns a value and pauses the execution** while maintaining the internal states, whereas the **return statement returns a value and terminates the execution** of the function.
- Generator function is a function which gives the next value when called repeatedly and saves memory.
- Generator are functions that returns a sequens of value. we use **yield** statement to return the from function.
- Yield statement returns the element from a generator function into a genrater object.(EX:- yield a)
- This function is used to retrieve element by element from a generator object.(Ex:- next(gen_obj))
- A generator is a special type of function which does not return a single value, instead, it returns an iterator object with a sequence of values.
- In a generator function, a **yield** statement is used rather than a return statement.
- The generator function cannot include the return keyword. If you include it, then it will terminate the function.

Features of Generator Function:

- Uses the yield keyword.
- Memory efficient → does not keep the entire list in memory.(Memory efficient होता है → पूरी list memory में नहीं रखता।)
- Can generate infinite series (infinite values).(Infinite series (अनंत values) generate कर सकता है।)
- Can be easily used with iteration (loop).(Iteration (loop) के साथ आसानी से इस्तेमाल किया जा सकता है।)

```
# Normal function
def normal_func():
    return [1, 2, 3, 4, 5]

# This will return the entire list at once.
print(normal_func())    # Output:- [1, 2, 3, 4, 5]

# Generator Function
def generator_func():
    for i in range(1, 6):
        yield i    # return नहीं, yield

# 🔄 यहाँ yield हर बार next() call करने पर अगली value return करेगा।
gen = generator_func()
print(next(gen))    # 1
print(next(gen))    # 2
print(next(gen))    # 3
```

- Generator with Loop

```
def squares(n):
    for i in range(1, n+1):
        yield i * i

for value in squares(5):
    print(value)
```

```
def mygenerator():
    print('First item')
    yield 10

    print('Second item')
    yield 20

    print('Last item')
    yield 30

gen = mygenerator()
print(next(gen))
print(gen.__next__()) # 2 option to write the next function
print(next(gen))
print(next(gen))
```

Output:-

First item

10

Second item

20

Last item

30

Traceback (most recent call last):

File "<string>", line 22, in <module>

StopIteration

2nd Option

```
gen = mygenerator()
while True:
    try:
        print ("Received on next(): ", next(gen))
    except StopIteration:
        break
```

Output:-

First item

Received on next(): 10

Second item

Received on next(): 20

Last item

Received on next(): 30

Example 2:-

```
def bhai(a,b):  
    yield a+b  
    yield a-b  
result = bhai(3,2)  
print(next(result))  
print(next(result))
```

Output:-

```
5  
1
```

Example 3:-

```
def numberPrint():  
    n = 1  
    while n <= 10:  
        sq = n*n  
        yield sq  
        n += 1  
values = numberPrint()  
for i in values:  
    print(i)
```

Output:-

```
1  
4  
9  
16  
25  
36  
49  
64  
81  
100
```